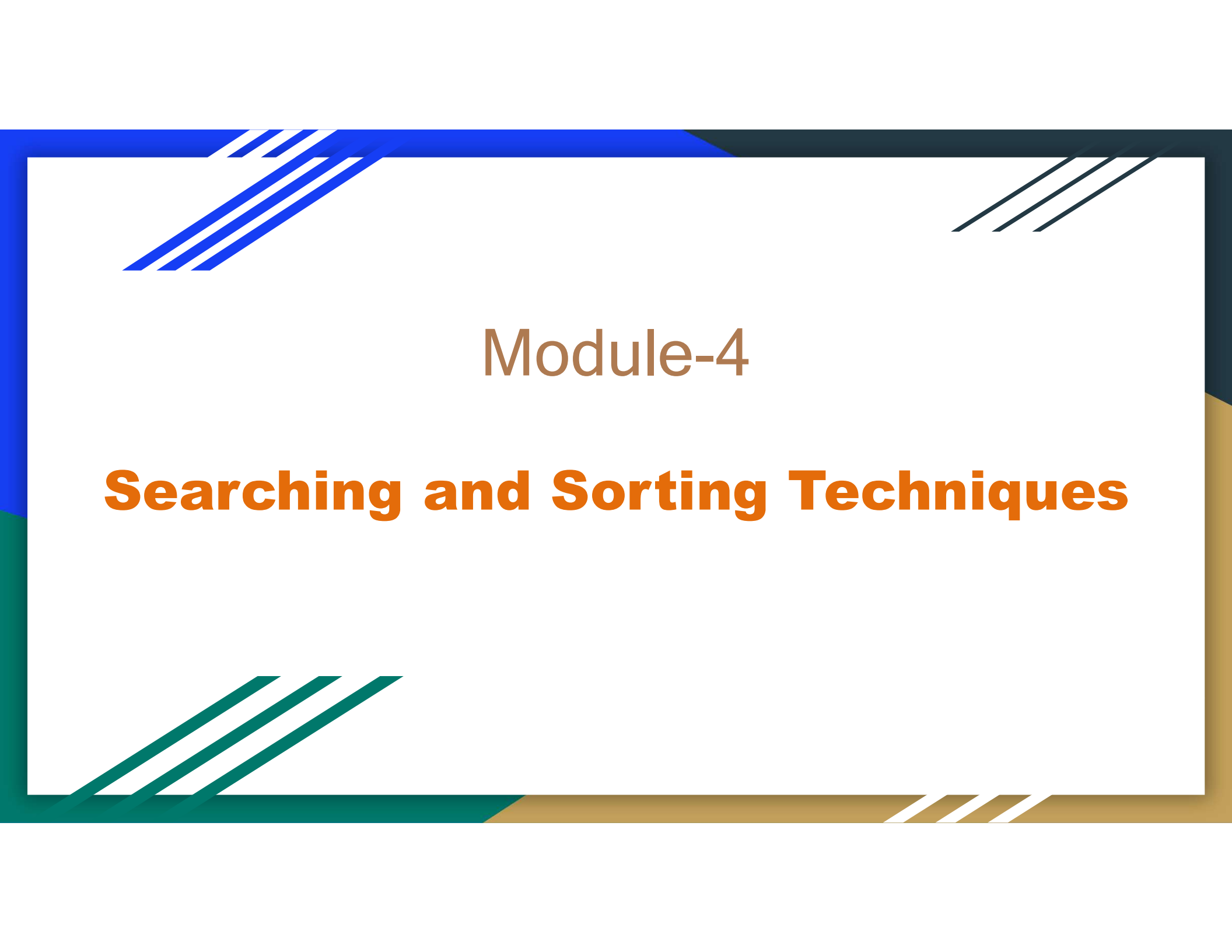




Module-4

Searching and Sorting Techniques

SEARCHING TECHNIQUES

Linear Search

Binary Search

LINEAR SEARCH

- It is also called as Sequential Search
- Searches for a particular value (*KEY*) in an array sequentially
- Compares *KEY* with every element in the array in a sequence until a match is found
- It is mostly used to search an unordered list of elements.

Example: A[]={1,22,33,55,44},

- key=55
- Comparisons:
 - 1==55 : F
 - 22==55:F
 - 33==55:F
 - 55==55: True =>Key found
- Key=7
- No. of comparisons:5
 - 1==7,22==7,33==7,55==7,44==7:False =>Key not found

LINEAR SEARCH ALGORITHM

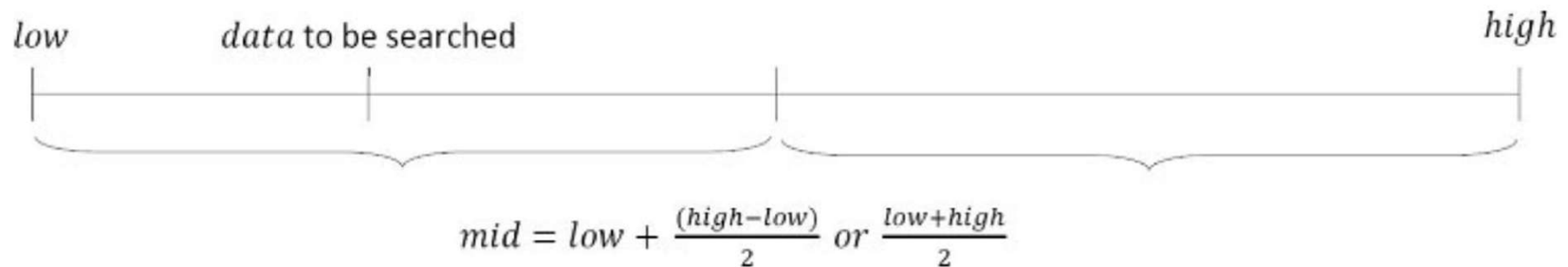
Algorithm LinearSearch(A[],key,n)

```
{  
for(i=0;i<n;i++){  
if (A[i]==key){  
    print("key found");break;}  
}  
if(i==n)  
    print("Key Not Found")  
}
```

Time complexity: $O(n)$

BINARY SEARCH

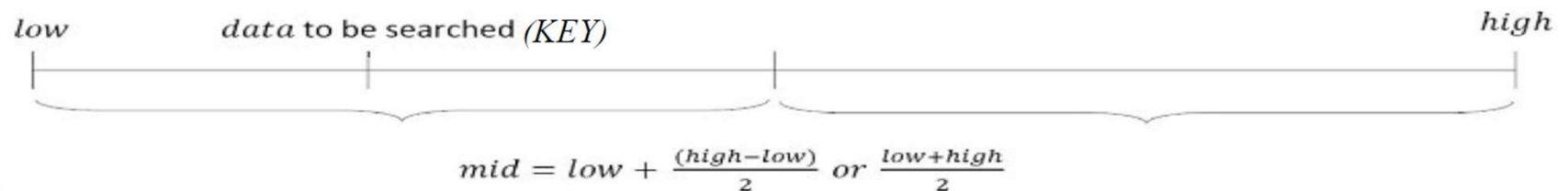
- Searches for a particular value (*KEY*) in a sorted array with run-time complexity of $O(\log n)$
- It works on the principle of Divide and Conquer
- Compares *KEY* with middle element(*mid*) in the array



BINARY SEARCH

In binary search,

1. If $KEY == A[mid]$, then search successful.
 2. If $KEY < A[mid]$, then KEY will be present in the left segment of the array. So, the value of $high$ will be changed as $high = mid - 1$.
 3. If $KEY > A[mid]$, then KEY will be present in the right segment of the array. So, the value of low will be changed as $low = mid + 1$.
- Finally, if KEY is not present in the array, then eventually, $high$ will be less than low . When this happens, the algorithm will terminate and the search will be unsuccessful.



BINARY SEARCH

Example: $A[] = \{1, 22, 33, 44, 55\}$, *Elements of array have to be in sorted order

- $\text{key} = 55$
- Comparisons: $\text{low} = 0, \text{mid} = 2, \text{high} = 4$
 - $55 == 33(A[2]): F$
 - $55 > 33: \text{True} \Rightarrow \text{low} = 3, \text{mid} = 3, \text{high} = 4$
 - $55 == 44(A[3]): F, 55 > 44 \Rightarrow \text{low} = 4, \text{mid} = 4, \text{high} = 4$
 - $55 == 55(A[4]): \text{True} \Rightarrow \text{Key found}$

BINARY SEARCH ALGORITHM

Algorithm BinarySearch(A[],key,n)

```
{
low=0;high=n-1;
mid=(low+high)/2;
while(low<=high){
    if (key==A[mid]){ print("key found");break;}
    else if (key<A[mid]){ high=mid-1;}
    else      low=mid+1;
    mid=(low+high)/2;
}
if(low>high) print("Key Not Found")
}
}
```

Time complexity: $O(\log n)$

Sorting

Sorting is an algorithm that arranges the elements of a list in a certain order [either ascending or descending].

The output is a permutation or reordering of the input.

Classification of Sorting Algorithms

Sorting algorithms are generally categorized based on the following parameters

- By number of comparisons
- By number of swaps
- By memory usage
- By recursion
- By Stability
- By Adaptability

Sorting Techniques

- Insertion sort
- Heap Sort
- Merge Sort
- Quick Sort

Insertion Sort

Idea: **like sorting a hand of playing cards**

Start with an empty left hand and the cards facing down on the table.

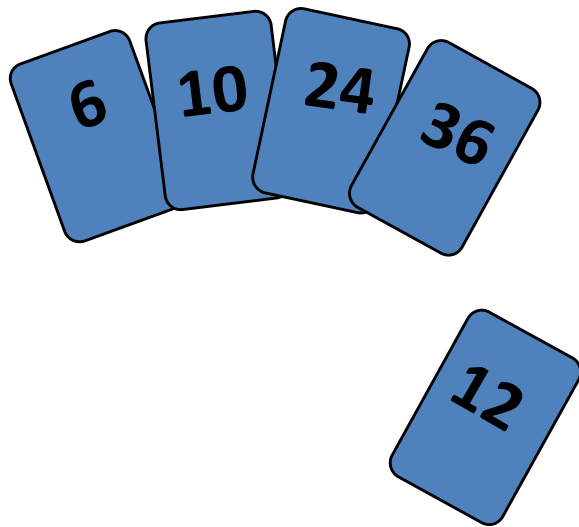
Remove one card at a time from the table, and insert it into the correct position in the left hand

compare it with each of the cards already in the hand, from right to left

The cards held in the left hand are sorted

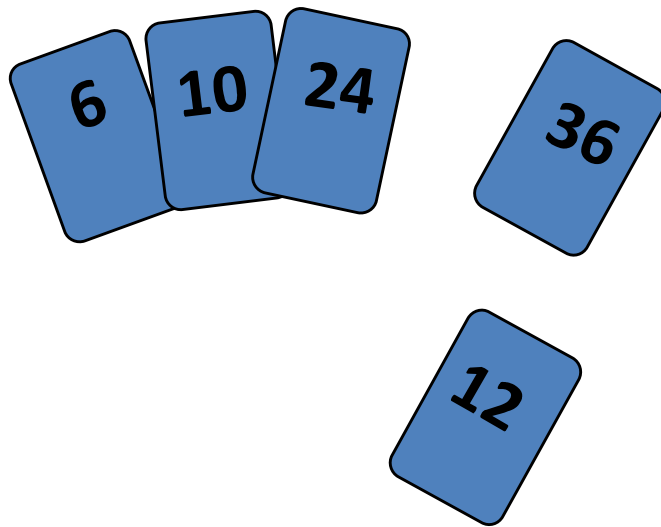
these cards were originally the top cards of the pile on the table

playing cards

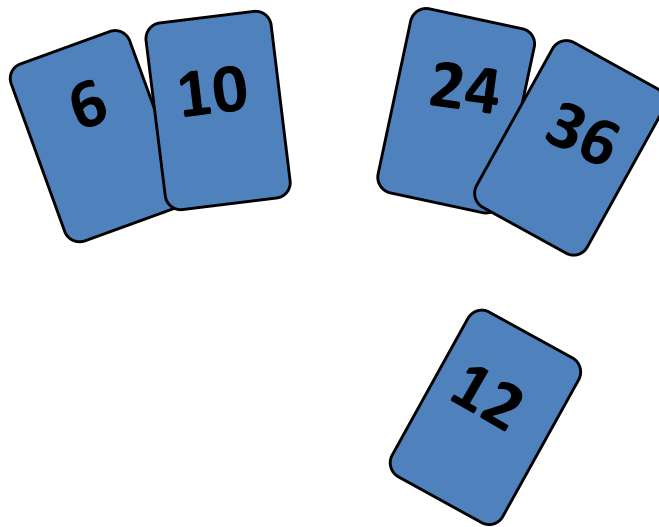


To insert 12, we need to make room for it by moving first 36 and then 24.

Insertion Sort

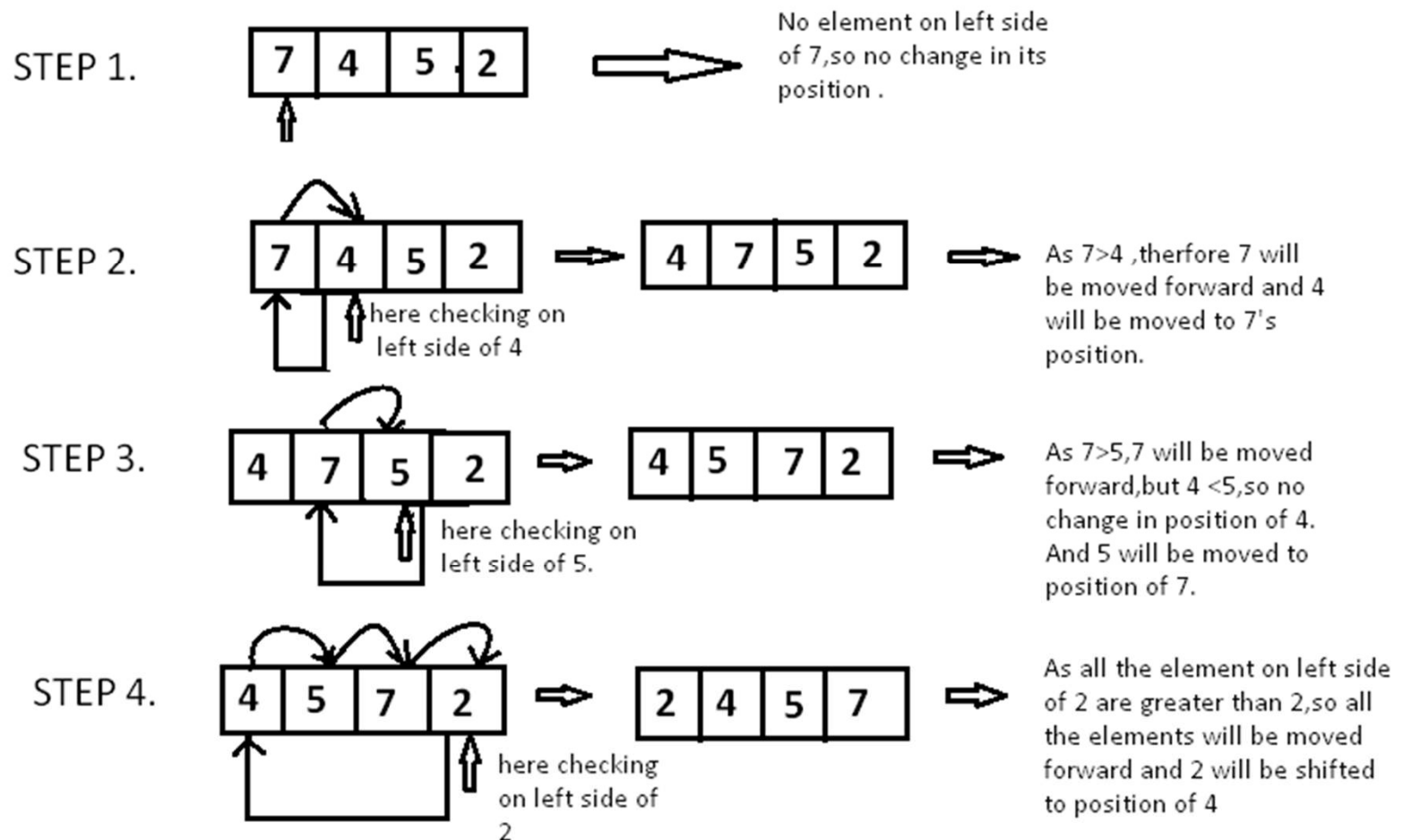


Insertion Sort



Try with an example: $A[] = \{7, 4, 5, 2\}$

Example1

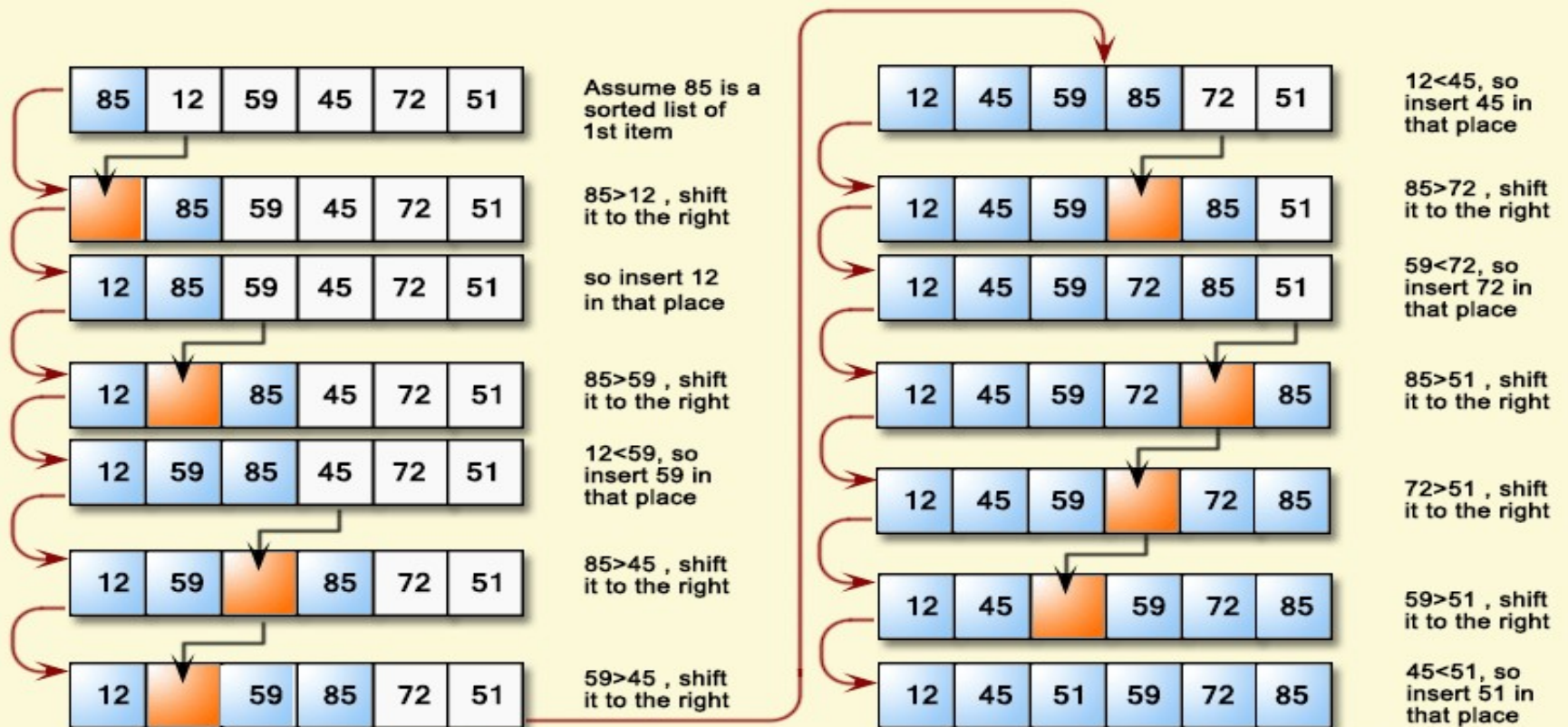


Algorithm:

```
Algorithm InsertionSort(A[], n)
{
    for(i=0;i<n-1;i++){
        /*storing current element whose left side is checked for its correct position*/
        temp:=A[i];  j:=i;
        /*check if the adjacent element in left side is greater or less than the current element*/
        while(j>0 && temp<A[j-1]){
            A[j]:=A[j-1]; j:=j-1;
        }
        A[j]:=temp;
    }
}
```

Example2

Insertion Sort



Code:

```
void insertion_sort ( int A[ ] , int n)
{
    for( int i = 0 ;i < n ; i++ ) {
        /*storing current element whose left side is checked for its
           correct position .*/

        int temp = A[ i ];
        int j = i;

        /* check whether the adjacent element in left side is greater or
           less than the current element. */

        while( j > 0  && temp < A[ j -1]) {

            // moving the left side element to one position forward.
            A[ j ] = A[ j-1];
            j= j - 1;

        }
        // moving current element to its correct position.
        A[ j ] = temp;
    }
}
```

Performance

Name Of Algorithm	Best Case Time	Average Case Time	Worst Case Time
Insertion Sort	$O(n)$	$\theta(n^2)$	$O(n^2)$

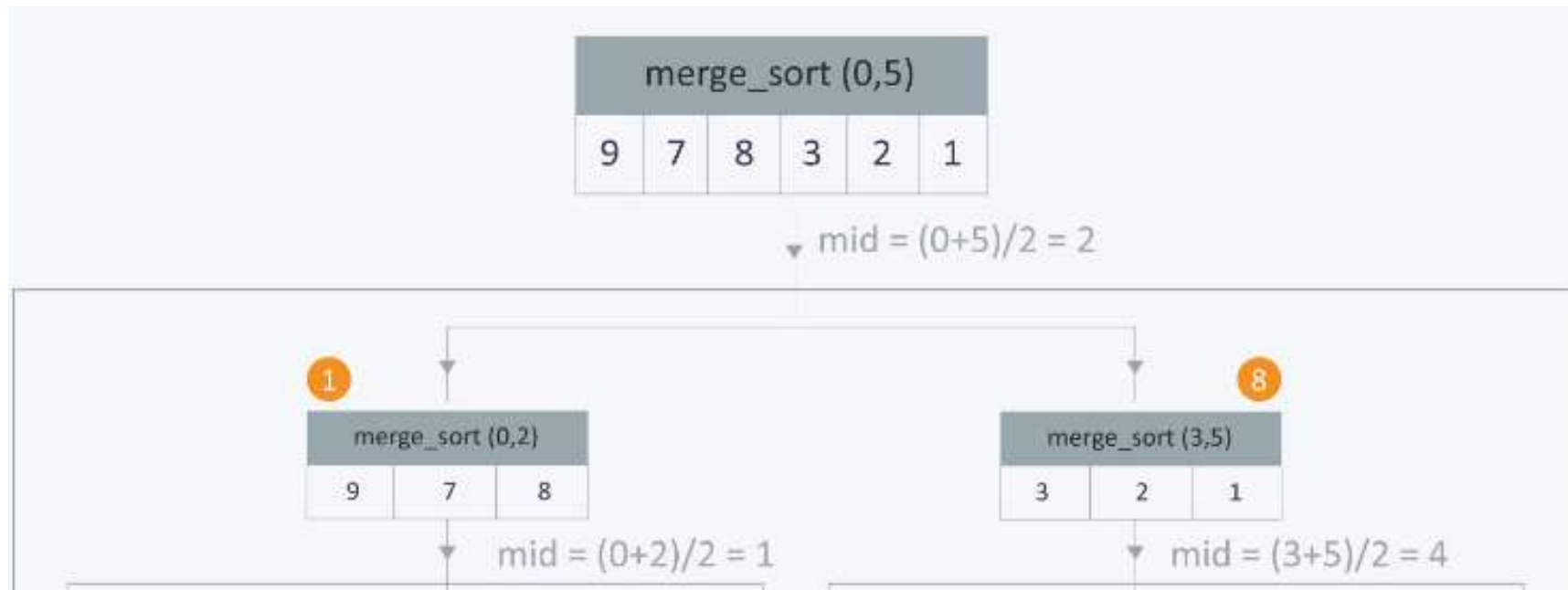
<https://www.cs.usfca.edu/~galles/visualization/ComparisonSort.html>

Merge sort

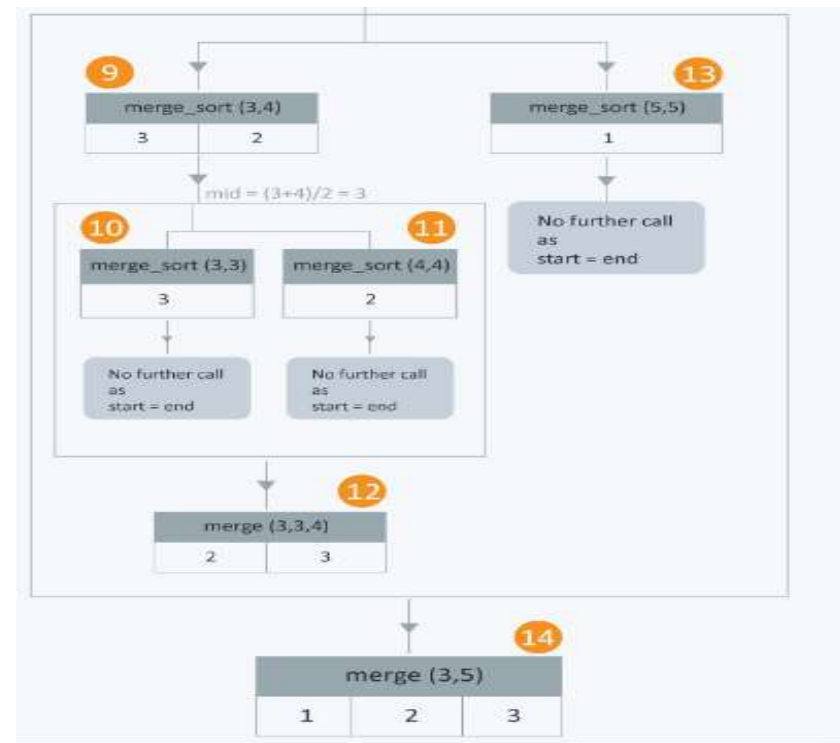
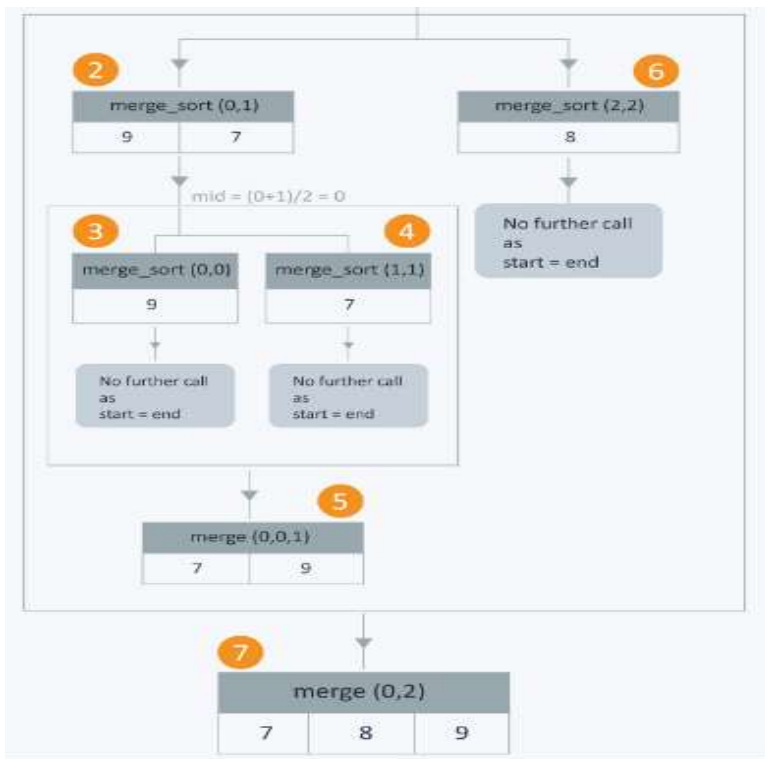
Merge sort is a divide-and-conquer algorithm based on the idea of breaking down a list into several sub-lists until each sublist consists of a single element and merging those sublists in a manner that results into a sorted list.

Idea:

- Divide the unsorted list into N sublists, each containing 1 element.
- Take adjacent pairs of two singleton lists and merge them to form a list of 2 elements. N will now convert into $N/2$ lists of size 2.
- Repeat the process till a single sorted list of obtained.



Example 1



Algorithm:

Algorithm Merge_Sort(A[], start, end)

{//start, end, mid are indices of A[]

if(start<end){

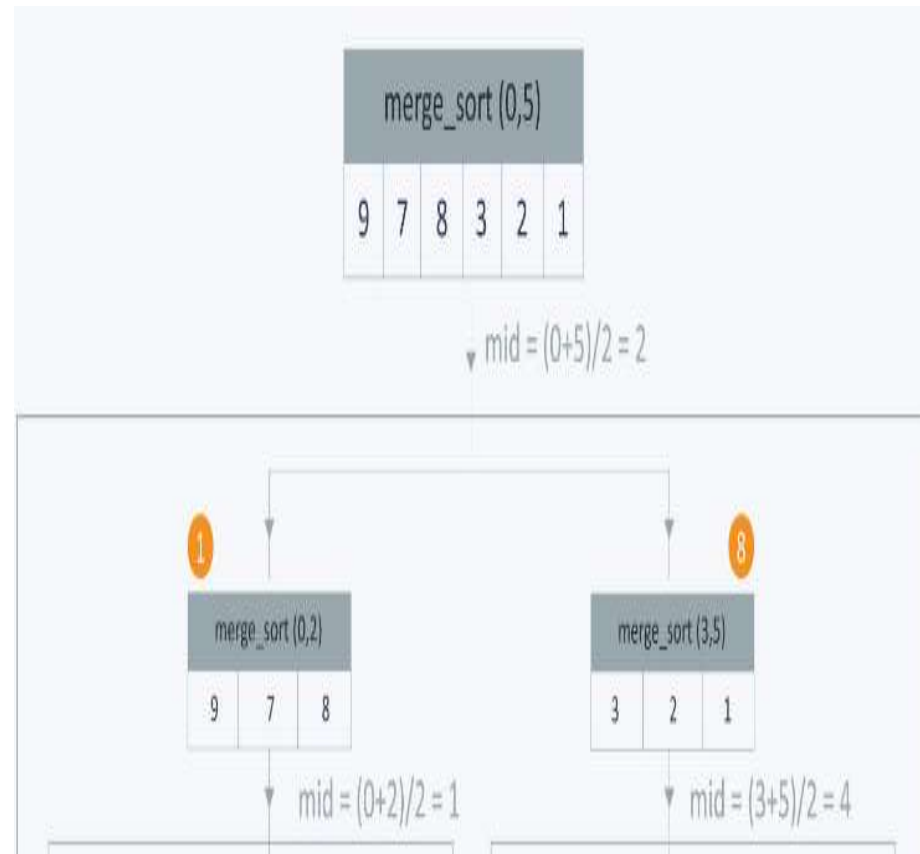
mid=(start+end)/2;

merge_sort(A, start, mid);

merge_sort(A, mid+1, end);

merge(A, start, mid, end);

}

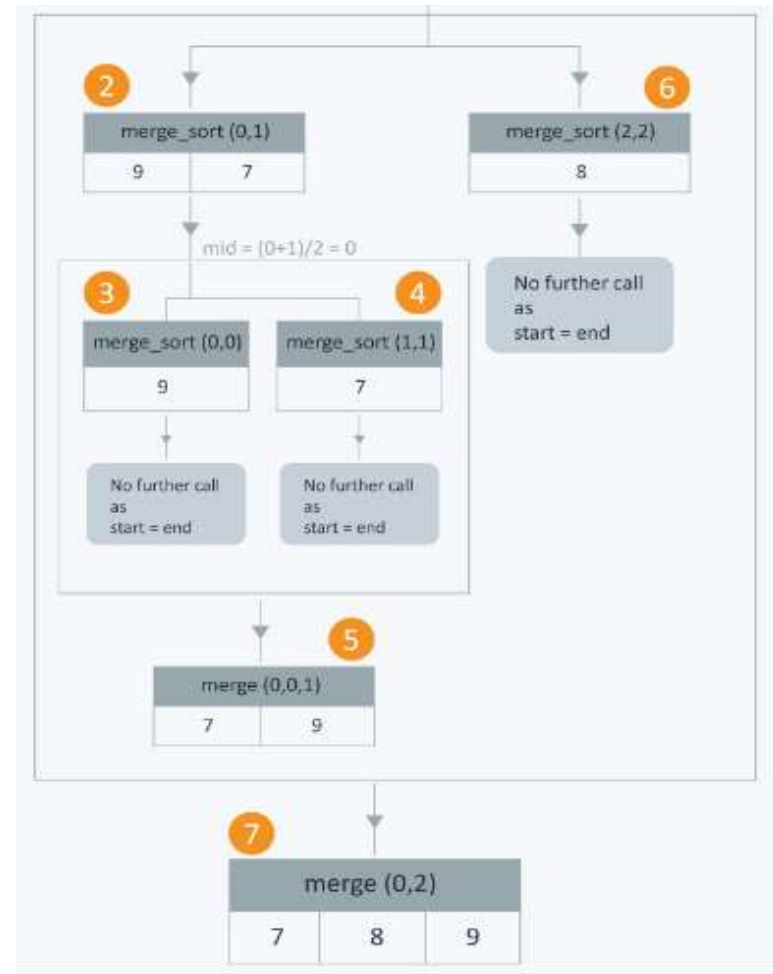


Algorithm:

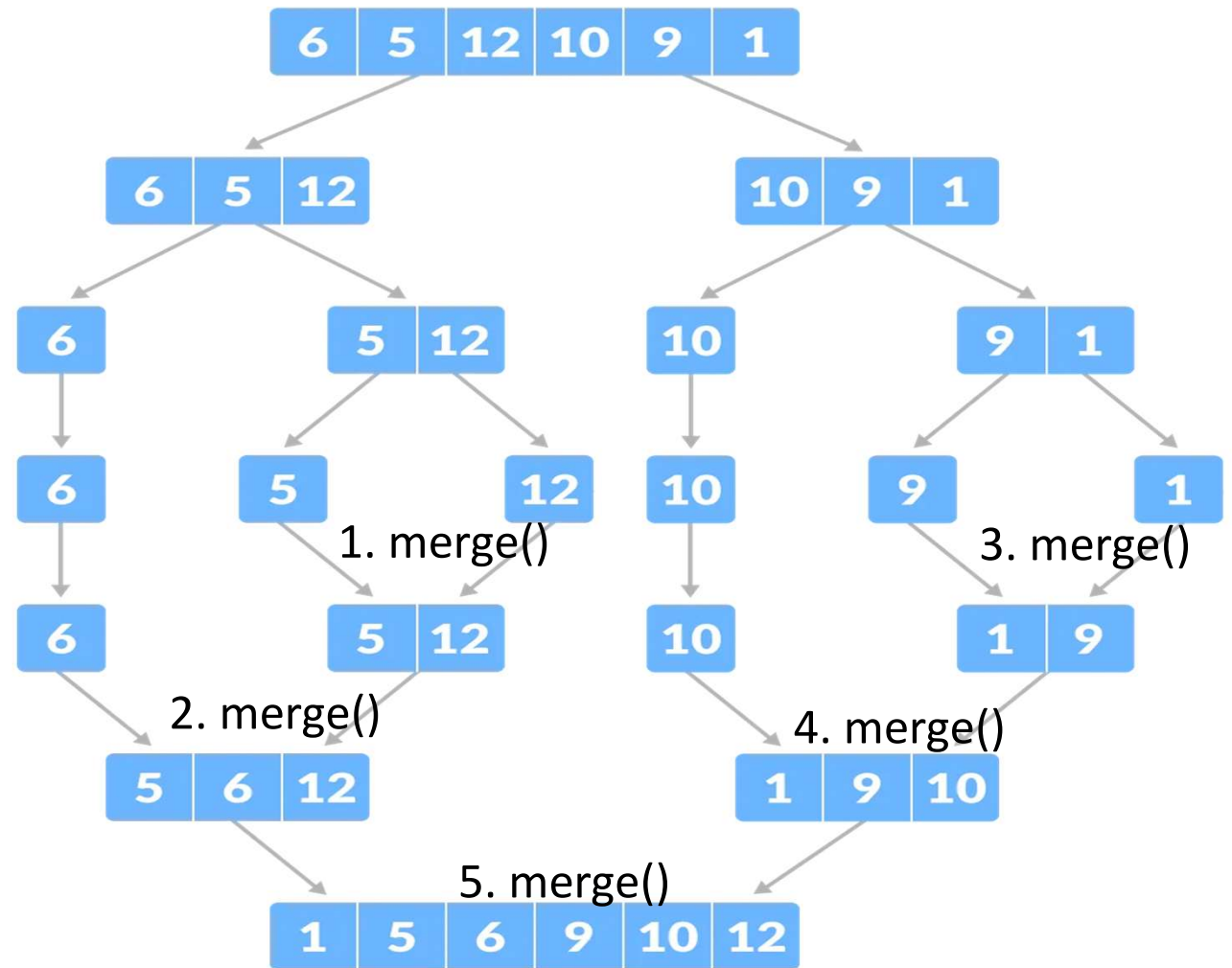
```

procedure merge(A[], start, mid, end)
{
  p=start; q=mid+1;
  k=0;
  for(i=start;i<=end;i++){
    if(p>mid)
      B[k++]=A[q++];
    else if(q>end)
      B[k++]=A[p++];
    else if(A[p]<A[q])
      B[k++]=A[p++];
    else
      B[k++]=A[q++];
  }
  for(p=0; p<k; p++) A[start++]=B[p];
}

```



Example1



Code:

```
void merge_sort (int A[ ], int start , int end )
{
    if( start < end ) {
        int mid = (start + end ) / 2 ;           // defines the current array in 2 parts
        .
        merge_sort (A, start , mid ) ;           // sort the 1st part of array .
        merge_sort (A,mid+1 , end ) ;           // sort the 2nd part of array.

        // merge the both parts by comparing elements of both the parts.
        merge(A,start , mid , end );
    }
}
```

Code: (Cont..)

```
void merge(int A[ ] , int start, int mid, int end) {
    //stores the starting position of both parts in temporary variables.
    int p = start ,q = mid+1;

    int Arr[end-start+1] , k=0;

    for(int i = start ;i <= end ;i++) {
        if(p > mid)          //checks if first part comes to an end or not .
            Arr[ k++ ] = A[ q++ ] ;

        else if ( q > end)    //checks if second part comes to an end or not
            Arr[ k++ ] = A[ p++ ];

        else if( A[ p ] < A[ q ])    //checks which part has smaller element.
            Arr[ k++ ] = A[ p++ ];

        else
            Arr[ k++ ] = A[ q++];
    }
    for (int p=0 ; p< k ;p ++ ) {
        /* Now the real array has elements in sorted manner including both
           parts.*/
        A[ start++ ] = Arr[ p ] ;
    }
}
```

Performance

Name Of Algorithm	Best Case Time	Average Case Time	Worst Case Time
Insertion Sort	$O(n)$	$\theta(n^2)$	$O(n^2)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$

<https://www.cs.usfca.edu/~galles/visualization/ComparisonSort.html>

Quick sort

Quick sort is based on the divide-and-conquer approach.

Idea:

- choosing one element as a pivot element and partitioning the array around it

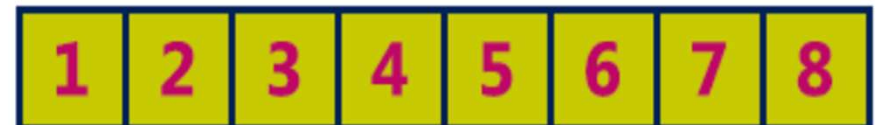
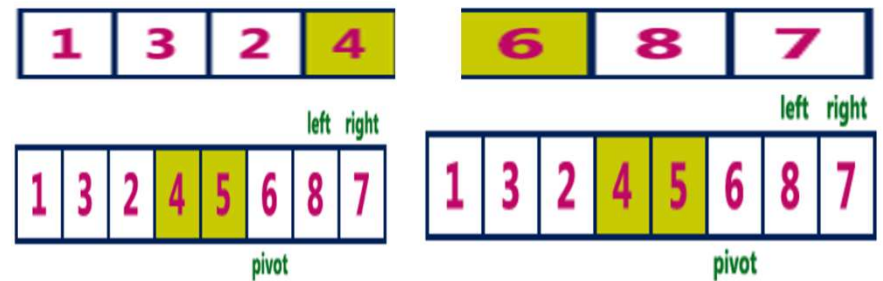
such that:

- Left side of pivot contains all the elements that are less than the pivot element
- Right side contains all elements greater than the pivot

Implementation :

- Step 1 - Consider the first element of the list as **pivot** (i.e., Element at first position in the list).
- Step 2 - Define two variables *left* and *right*. Set *left* and *right* to index of first and last elements of the list respectively.
- Step 3 - Increment *left* until $A[left] > \text{pivot}$ then stop.
- Step 4 - Decrement *right* until $A[right] < \text{pivot}$ then stop.
- Step 5 - If $left < right$ then exchange $A[left]$ and $A[right]$.
- Step 6 - Repeat steps 3,4 & 5 until $left > right$.
- Step 7 - Exchange the pivot element with $A[right]$ element.
- Step 8- Repeat steps 1 to 7 for subarray to left of pivot element.
- Step-9-Repeat steps 1 to 7 for subarray to right of pivot element.

Example1



Algorithm:

```
Algorithm Quick_Sort(A[], start, end)
{ //start, end, pivot_pos are indices of A[]
  if(start < end){
    pivot_pos = partition(A, start, end);
    quick_sort(A, start, pivot_pos - 1);
    quick_sort(A, pivot_pos + 1, end);
  }
}
```

Algorithm:

```
procedure partition(A[], start, end)
{
    piv=A[start]; //make first element as pivot
    for(left=start+1, right=end; left<right;)
    {
        while(A[left]<piv) left=left+1;
        while(A[right]>piv) right=right-1;
        if(left<right) swap(A[left], A[right]);
    }
    swap(A[start], A[right]);
    return right; // return position of pivot
}
```

Code:

```
void quick_sort ( int A[ ],int start , int end ) {  
    if( start < end ) {  
        //stores the position of pivot element  
        int piv_pos = partition (A,start , end ) ;  
        quick_sort (A,start , piv_pos -1);    //sorts the left side of pivot.  
        quick_sort ( A,piv_pos +1 , end) ; //sorts the right side of pivot.  
    }  
}
```

Code: (Cont..)

```
int partition ( int A[],int start ,int end) {
    int i = start + 1;
    int piv = A[start] ;           //make the first element as pivot element.
    for(int j =start + 1; j <= end ; j++ ) {
        /*rearrange the array by putting elements which are less than pivot
        on one side and which are greater that on other. */

        if ( A[ j ] < piv) {
            swap (A[ i ],A [ j ]);
            i += 1;
        }
    }
    swap ( A[ start ] ,A[ i-1 ] ) ; //put the pivot element in its proper place.
    return i-1;                    //return the position of the pivot
}
```

Lets try to understand the pseudo code with an example: $A [] = \{ 7, 4, 5, 2 \}$

Performance

Name Of Algorithm	Best Case Time	Average Case Time	Worst Case Time
Insertion Sort	$O(n)$	$\theta(n^2)$	$O(n^2)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$

<https://www.baeldung.com/cs/quicksort-time-complexity-worst-case>

Binary heap

- A binary heap is a complete binary tree with some special properties.
- *Heap property*: The value of node must be $\geq$ (or $\leq$) to the values of its children.
- Types
 - Min heap ($\leq$)
 - Max heap ($\geq$)

Types of heaps

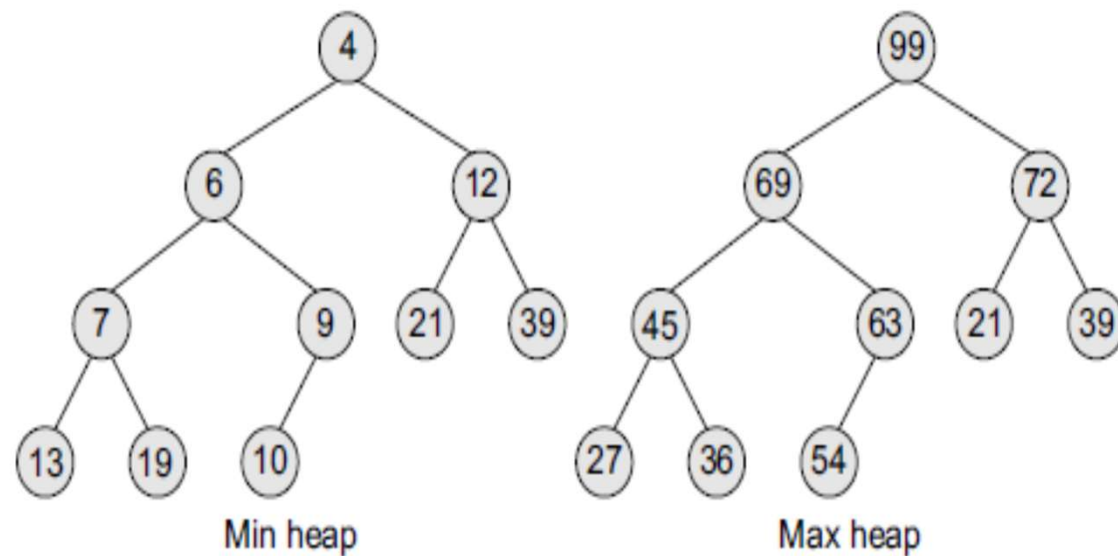


Figure 12.1 Binary heaps

Representation of Binary Heaps: Arrays

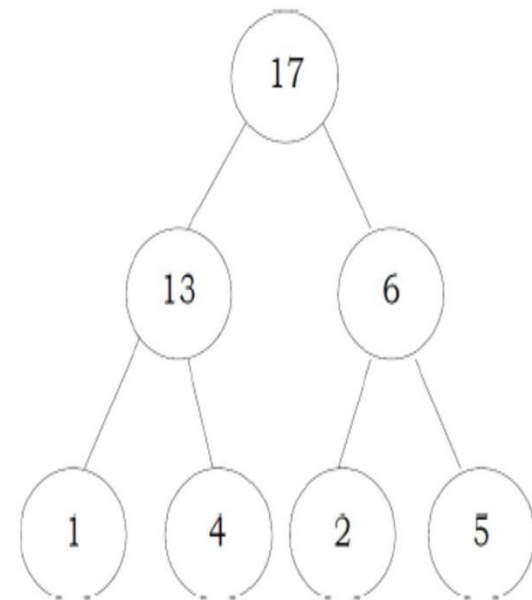
Root present at index 0

Parent index = i

Left child index = $2*i+1$

Right child index = $2*i+2$

17	13	6	1	4	2	5
0	1	2	3	4	5	6

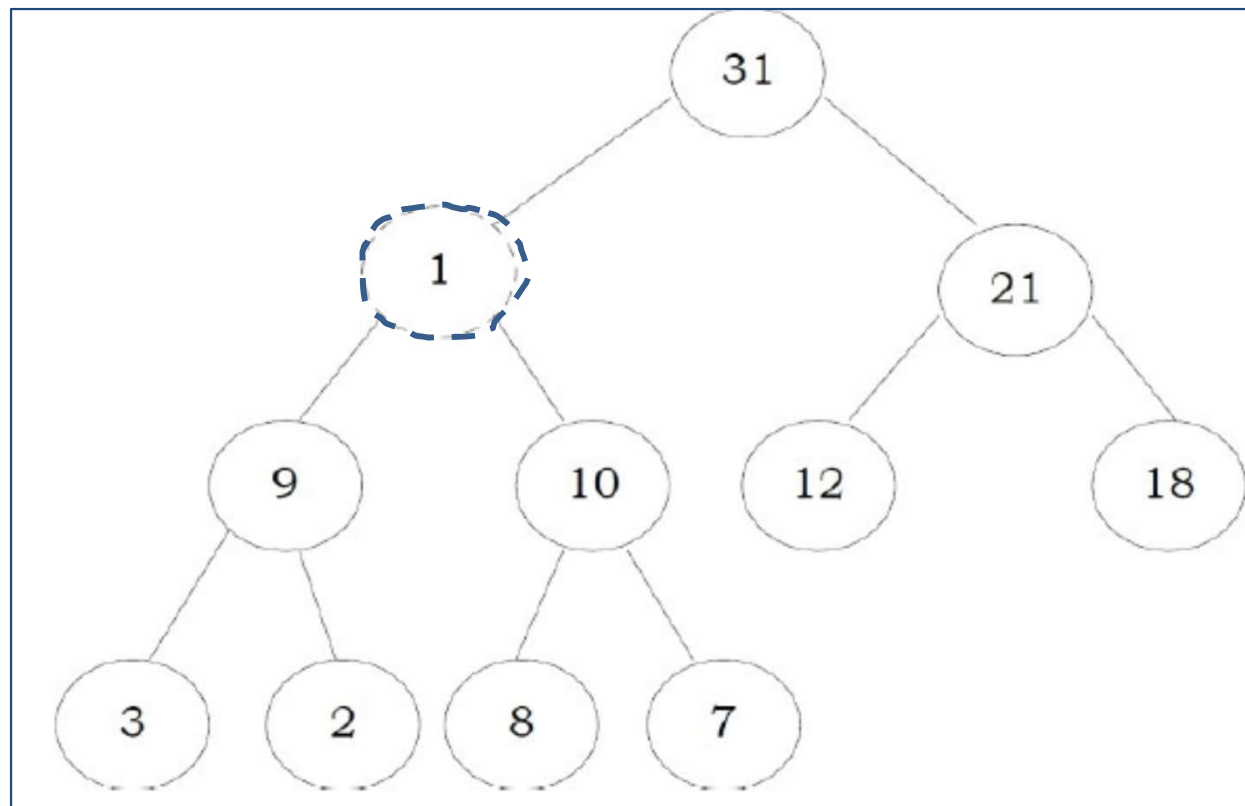


Heapifying an element

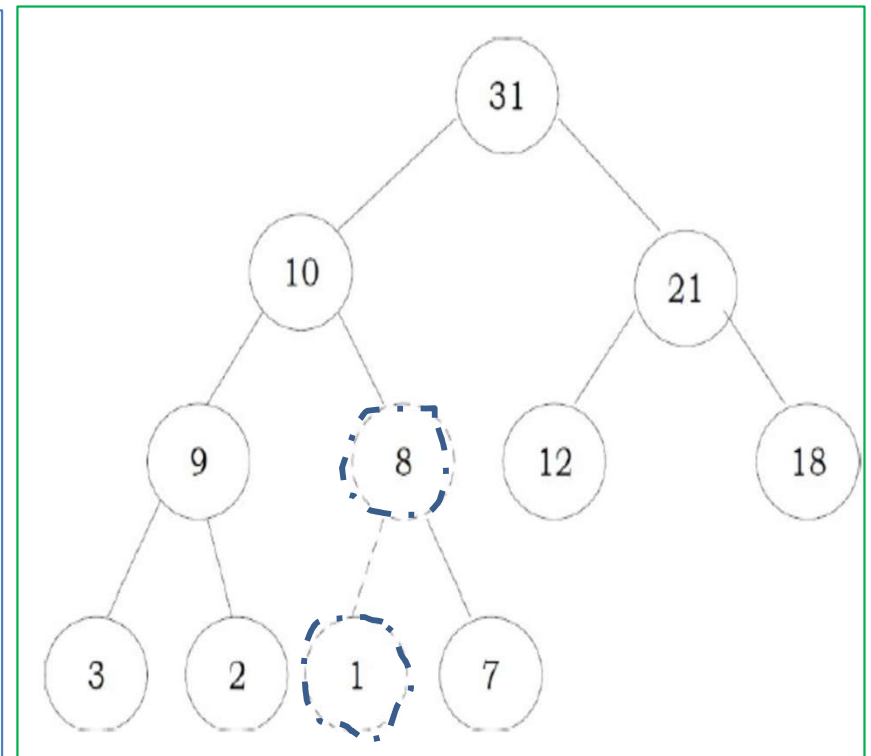
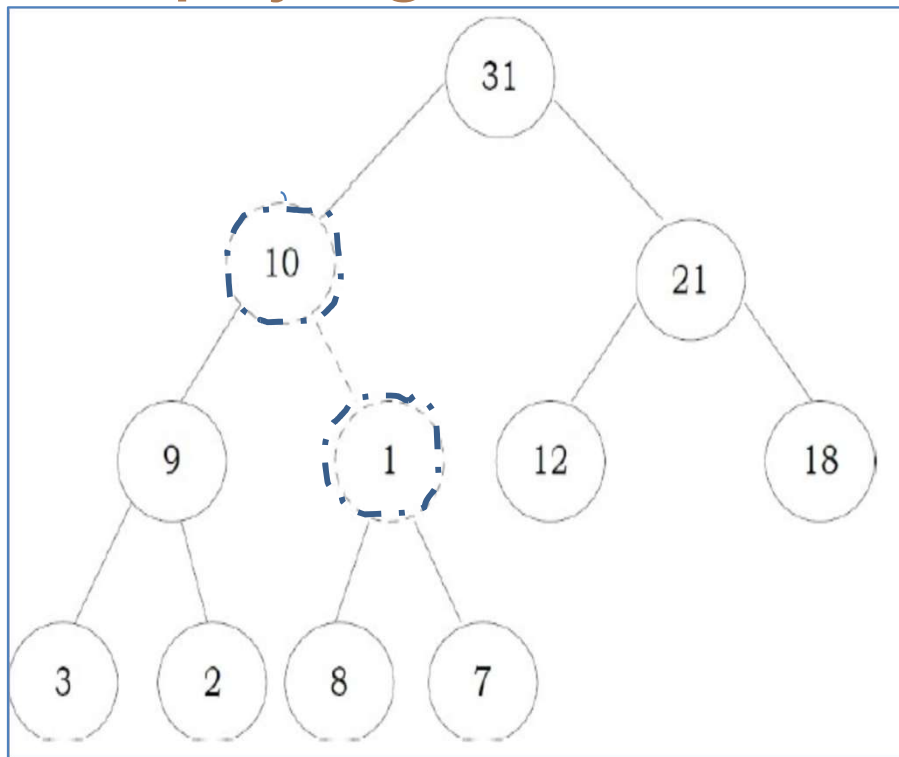
After inserting an element into heap, it may not satisfy the heap property. In that case we need to adjust the locations of the heap to make it heap again. This process is called *heapifying*.

To heapify an element in a max heap, find the maximum of its children and replace the element with that maximum value.

Heapifying an element



Heapifying an element



Heapifying an element

```
heapify(arr[],i, n)
{
    l=lchild(arr,i,n);
    r=rchild(arr,i,n);
    max=i;
    if(l!=-1&&arr[l]>arr[i])
        max=l;
    if(r!=-1&&arr[r]>arr[max])
        max=r;
    if(max=i) return;
    swap(arr,i,max);
    heapify(arr,max,n);
}
```

Deleting an element in heap

1. Replace the root node's value with the last node's value so that H is still a complete binary tree but not necessarily a heap.
2. Delete the last node.
3. Sink down the new root node's value so that H satisfies the heap property (***Heapify***).

Deleting an element in heap

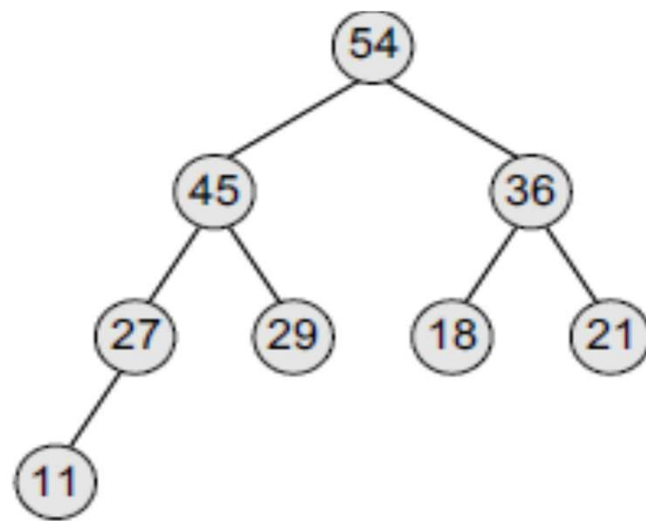
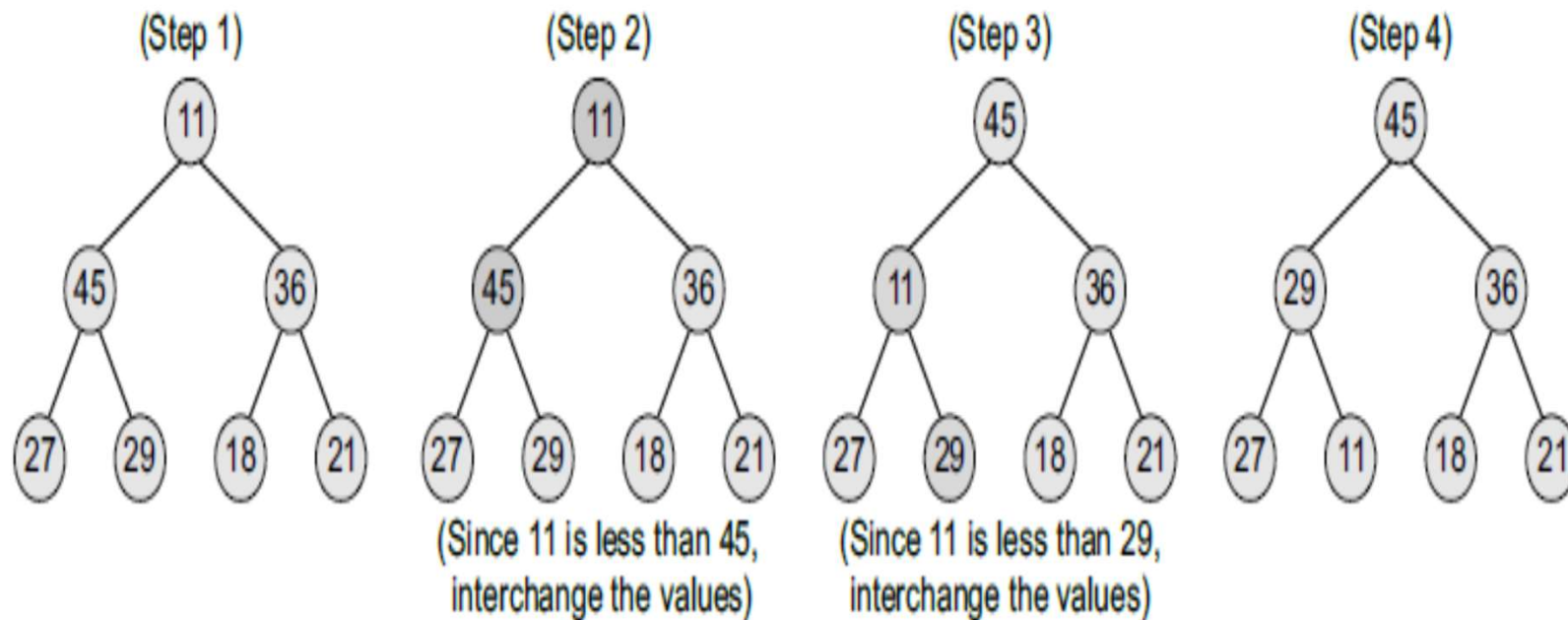


Figure 12.8 Binary heap

Deleting an element in heap



Inserting an element in heap

1. Add the new value at the bottom of H in such a way that H is still a complete binary tree but not necessarily a heap.
2. Let the new value rise to its appropriate place in H so that H now becomes a heap as well.

To do this, compare the new value with its parent to check if they are in the correct order. If they are, then the procedure halts, else the new value and its parent's value are swapped and Step 2 is repeated.

Inserting an element in heap

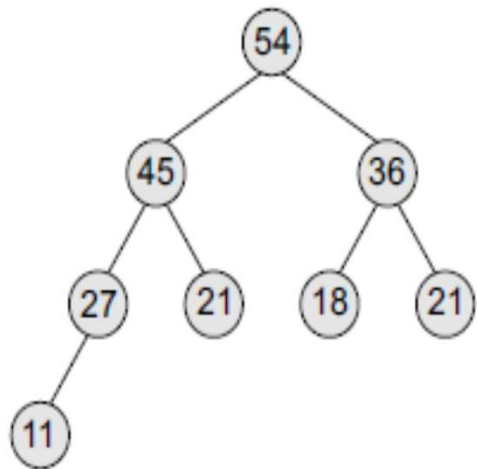


Figure 12.2 Binary heap

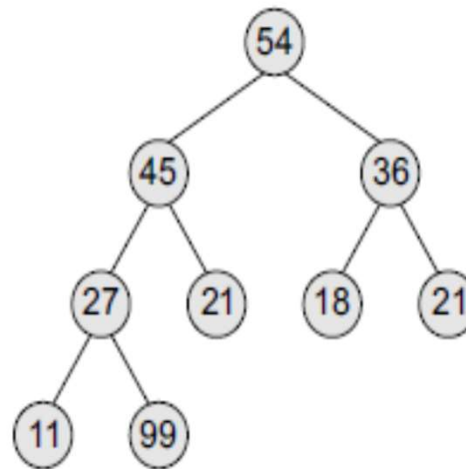


Figure 12.3 Binary heap after insertion of 99

Inserting an element in heap

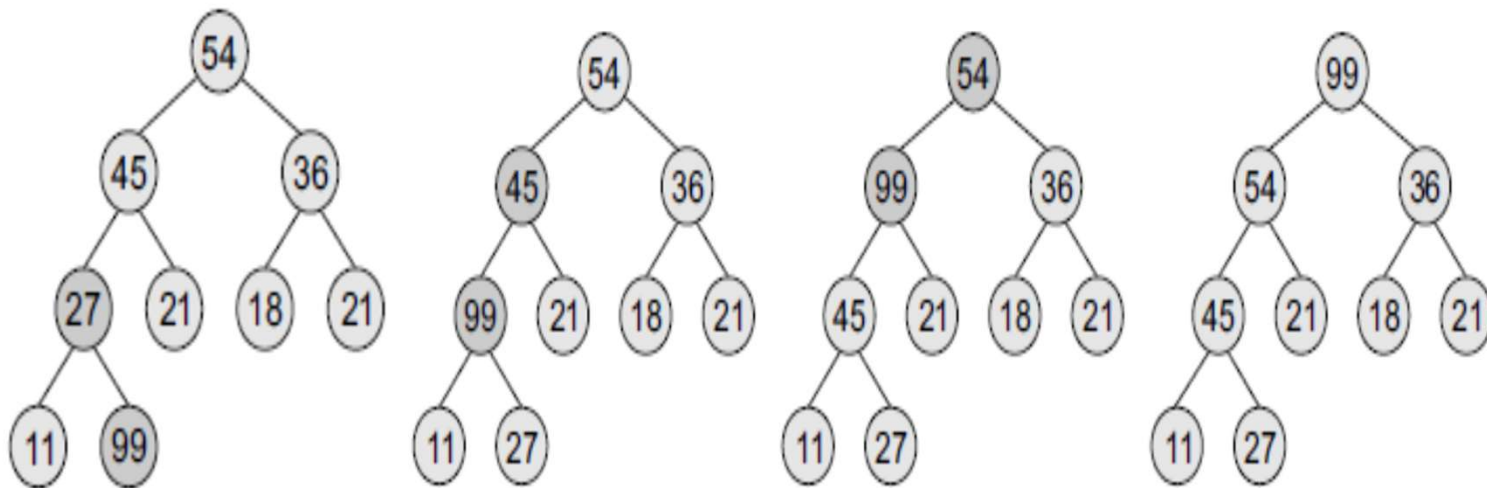


Figure 12.4 Heapify the binary heap

Build a heap

```
buildheap(arr[], n)
{
    for(i=(n-1)/2;i>=0;i--)
        heapify(arr,i,n);
}
```

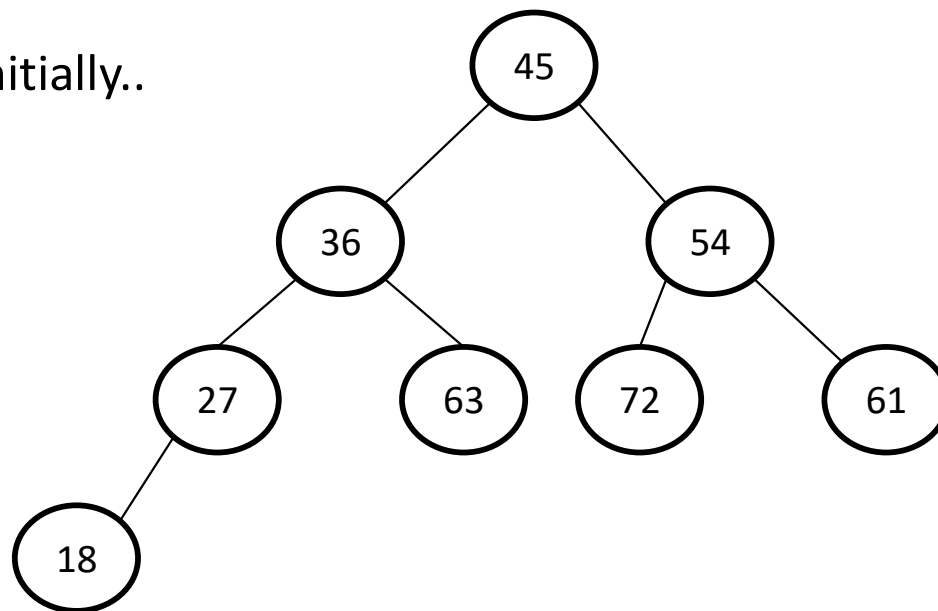
Build a heap

Build a max heap H from the given set of numbers:
45, 36, 54, 27, 63, 72, 61, and 18.

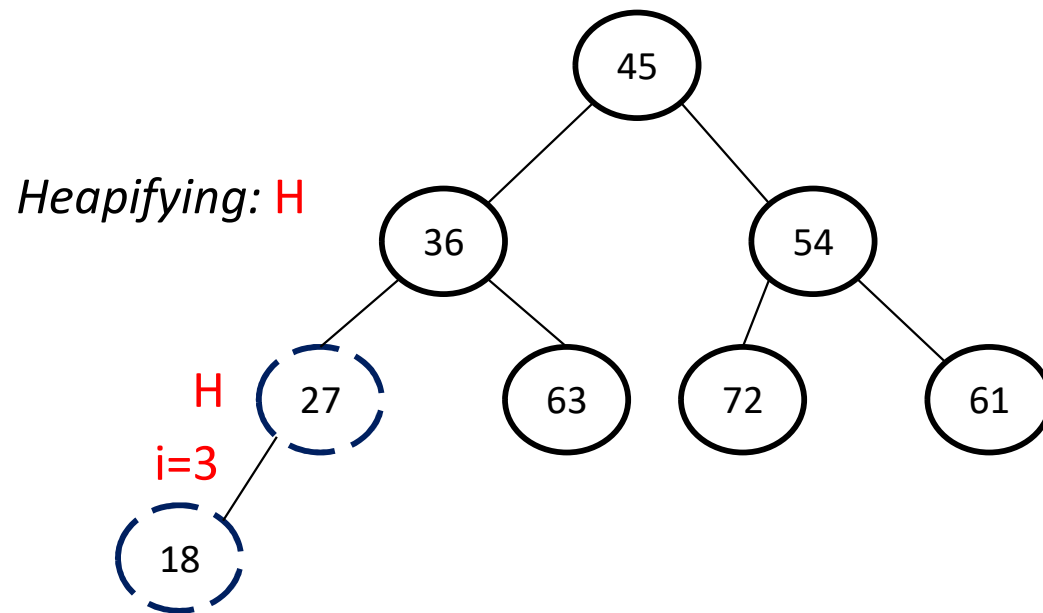
45	36	54	27	63	72	61	18
----	----	----	----	----	----	----	----

Build a heap: Example

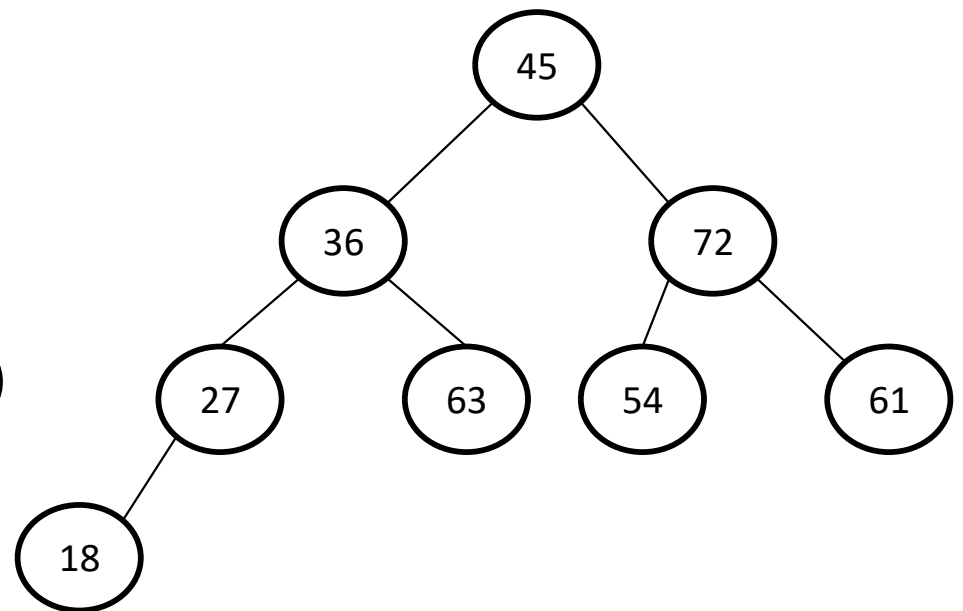
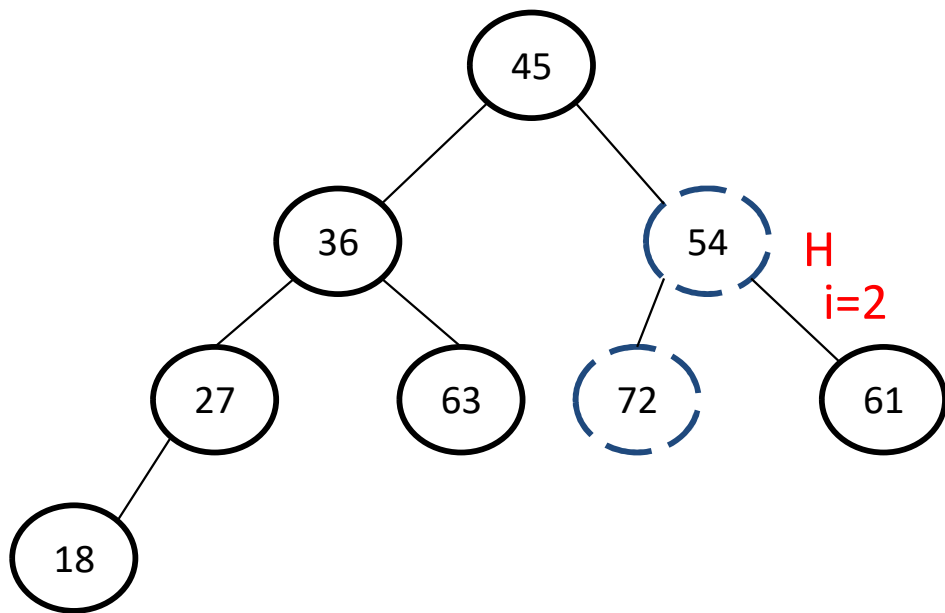
Initially..



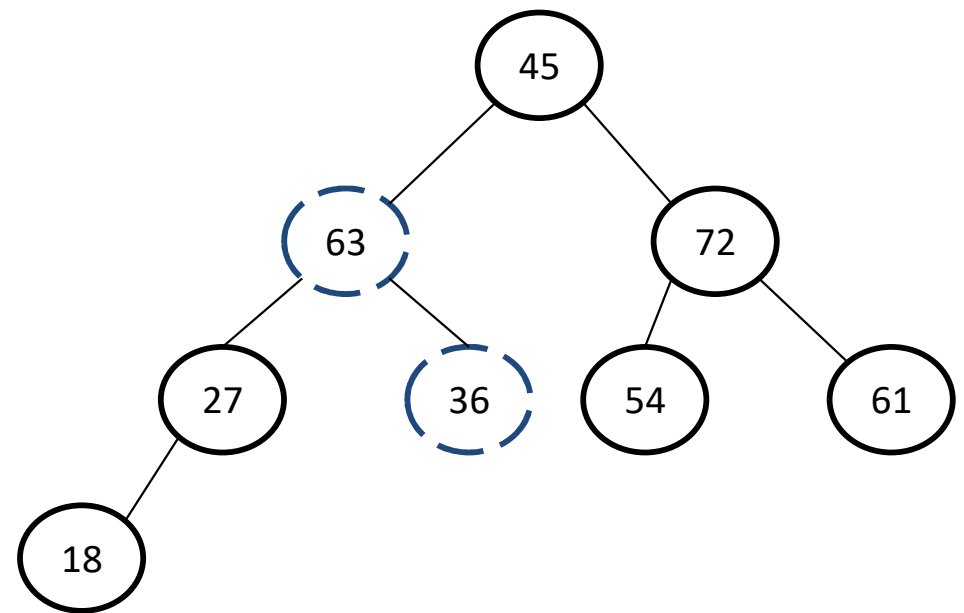
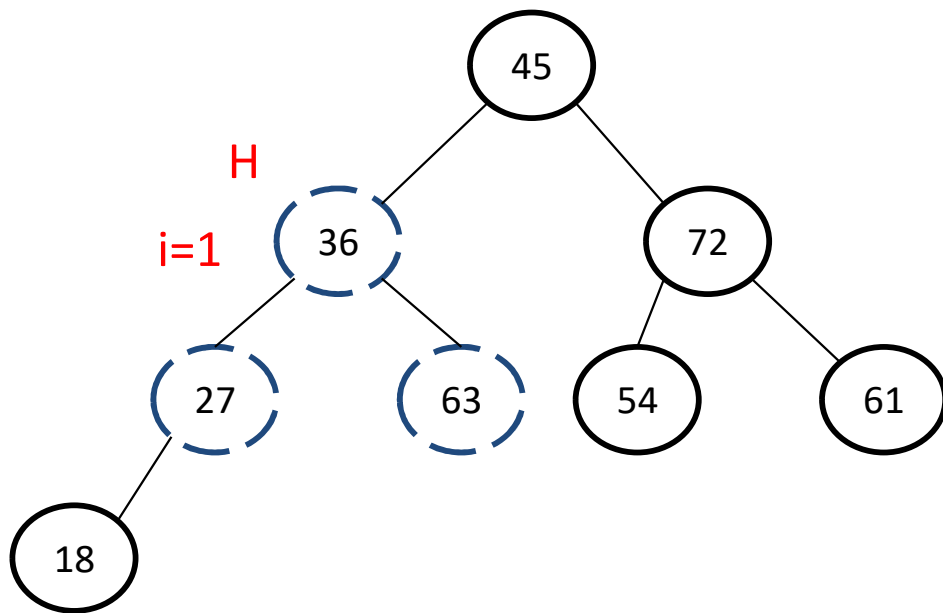
..Contd.



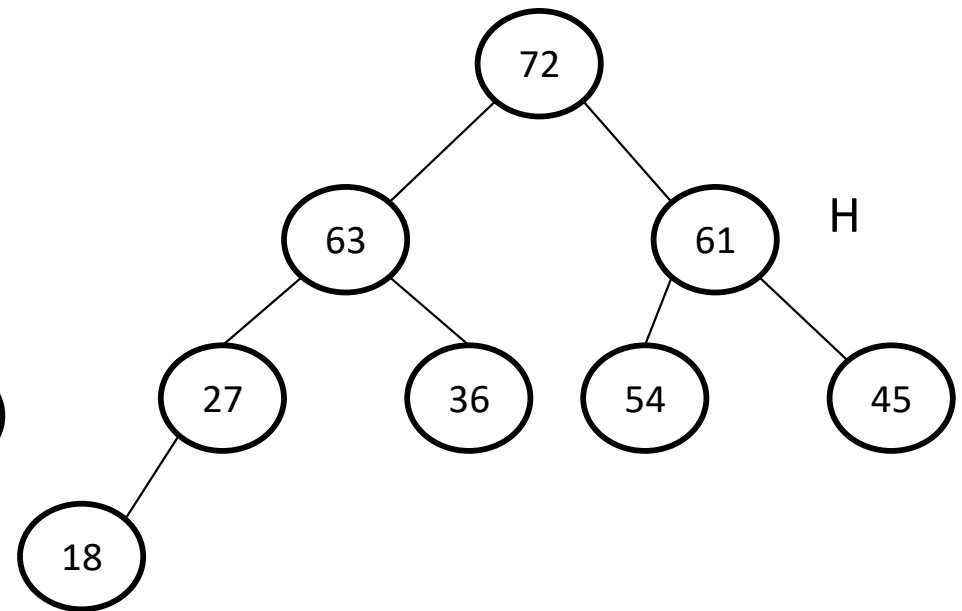
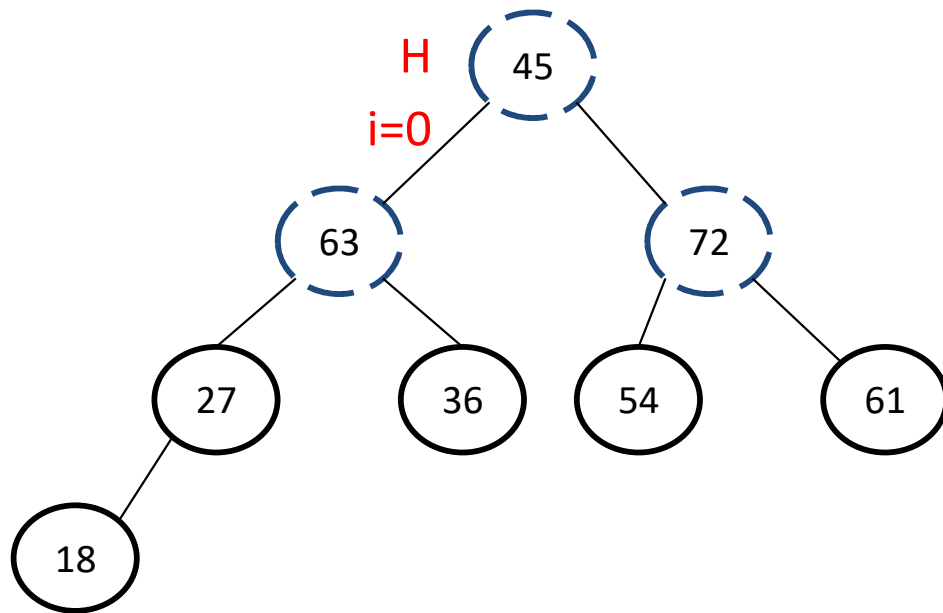
..Contd.



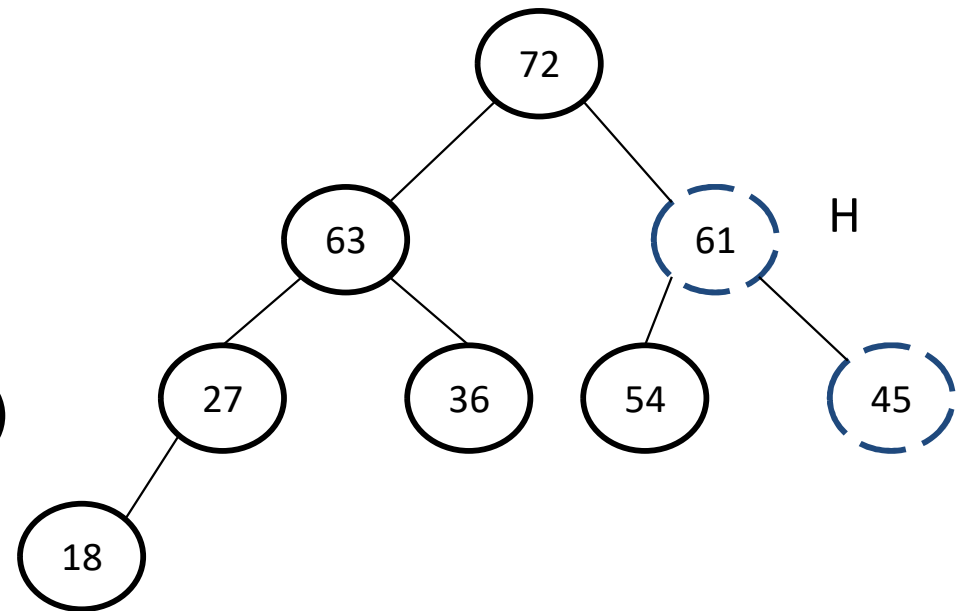
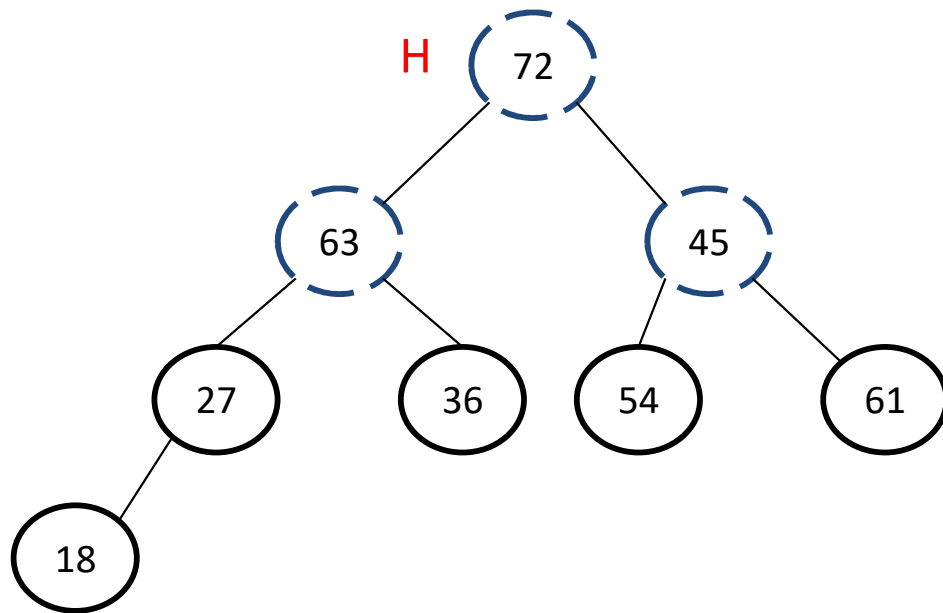
..Contd.



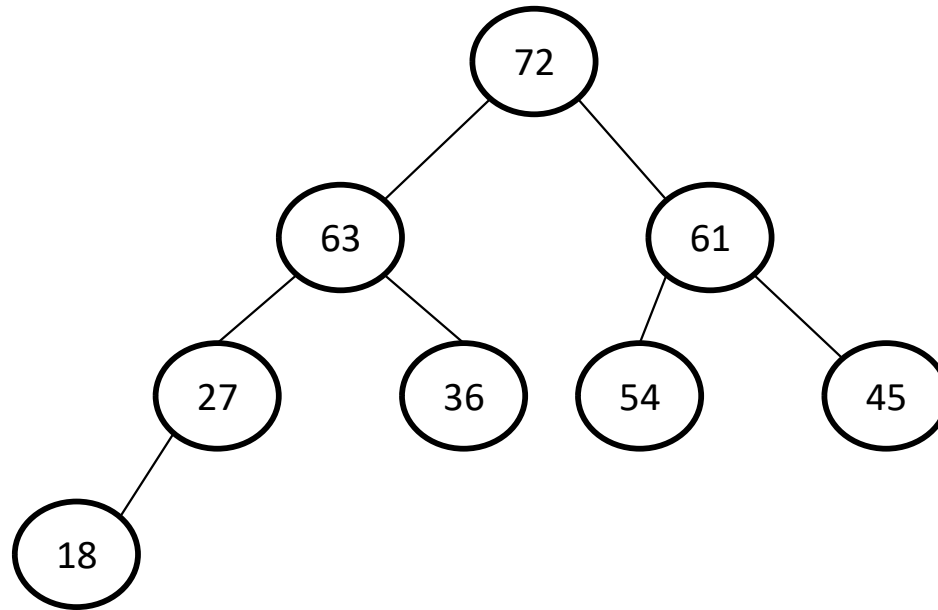
..Contd.



..Contd.



Binary heap :Example



72	63	61	27	36	54	45	18
----	----	----	----	----	----	----	----

Array representation of binary heap

Applications of heaps

- Priority Queue implementation
- Heap Sort

Heap Sort

- Two phases:
 1. Build a binary Heap/binary heap tree using given array elements
 2. Repeatedly delete the root element of binary heap tree of phase#1

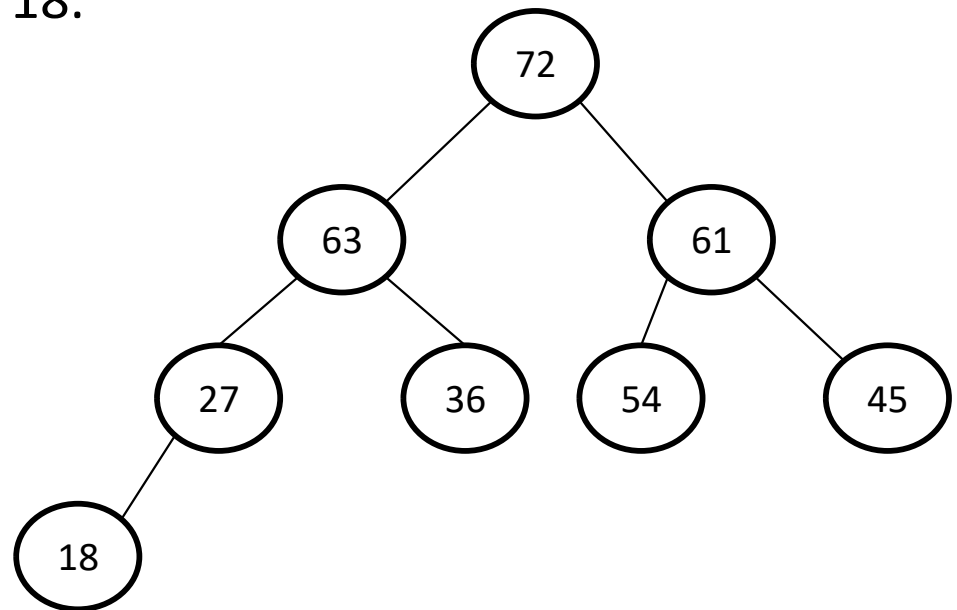
Heap Sort Pseudocode

```
procedure heapsort(arr, n)
{
    buildheap(arr,n);
    count=n;
    for(i=n-1;i>0;i--)
    {
        swap(arr,0,count);
        count--;
        heapify(arr,0,count);
    }
}
```

Heap Sort

Sort 45, 36, 54, 27, 63, 72, 61, 18.

#1: Build Heap



Heap Sort

Array: 72,63,61,27,36,54,45,18

#2: Delete root node

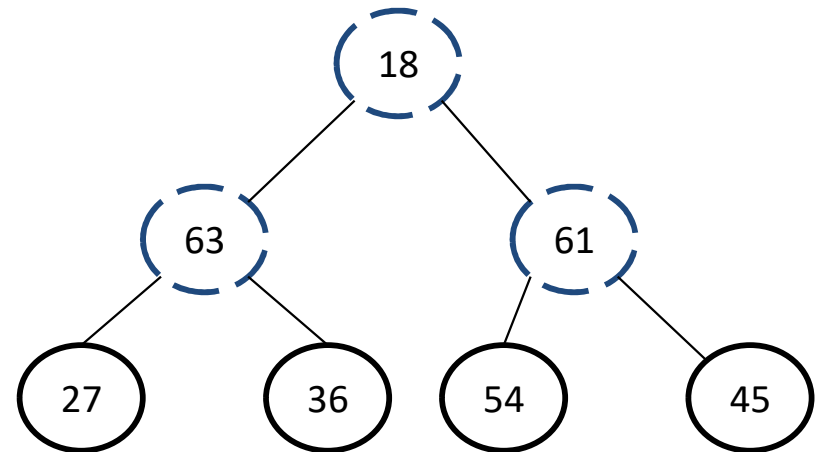
*Replace it with last node value and
heapify*

Repeat Phase #2

Array: 18,63,61,27,36,54,45,72

Array: 18,63,61,27,36,54,45

Array: 63, 36,61,27,18,54,45



Heap Sort

Repeat Phase #2

Array: 18, 63, 61, 27, 36, 54, 45

Array: 63, 36, 61, 27, 18, 54, 45

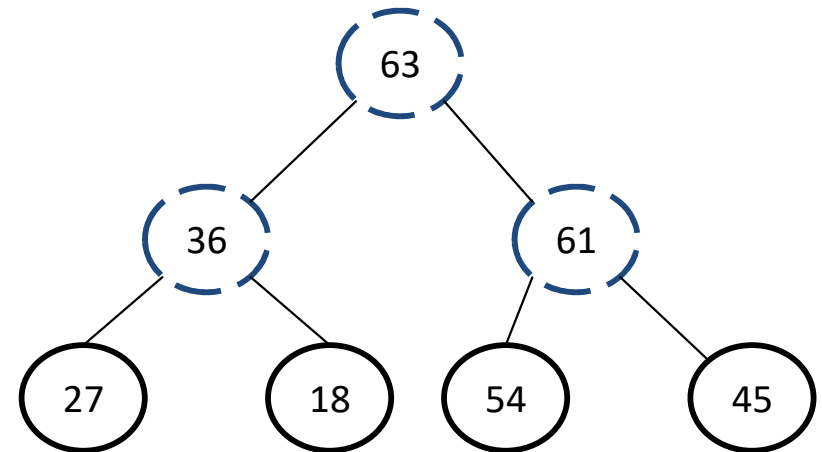
Delete 63

Array: 45, 36, 61, 27, 18, 54, 63

Array: 61, 36, 45, 27, 18, 54

Delete 61

Array: 54, 36, 45, 27, 18, 61,



Heap Sort

Repeat Phase #2

Array: 54, 36, 45, 27, 18

Delete 54

Array: 18, 36, 45, 27, 54

Array: 45, 36, 18, 27

Delete 45

Array: 27, 36, 18, 45

Array: 36, 27, 18

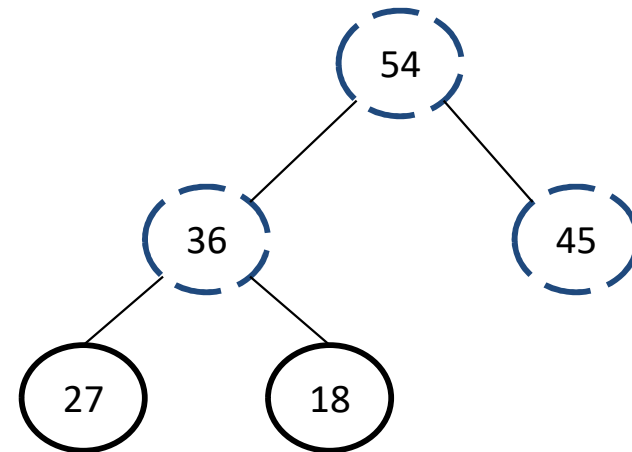
Delete 36

Array: 18, 27, 36, 45

Array: 27, 18

Delete 27

Final Array: 18, 27, 36, 45, 54, 61, 63, 72



Performance

Name Of Algorithm	Best Case Time	Average Case Time	Worst Case Time
Heap Sort	$\theta(n \log n)$	$\theta(n \log n)$	$\theta(n \log n)$

Heap Sort

Example 2: Create Heap for the following elements
and then sort them.

{13,102,405,136,15,105,390,432,28,444}

.